

Classification, Segmentation, and GAN-Based Augmentation for Concrete Crack Detection

Hrithik Mhatre

Dept. of Civil Engineering

Vamshika Sutar

Dept. of Civil Engineering

Prof. Abir De

Dept. of Computer Science and Engineering

Prof. Alankar Alankar

Dept. of Mechanical Engineering

Indian Institute of Technology Bombay

Mumbai – 400076, India

Abstract—Early detection of surface cracks is essential for ensuring the safety and longevity of civil infrastructure. Traditional manual inspection methods are subjective and error-prone, whereas deep learning offers a scalable and consistent alternative. However, real-world datasets often suffer from class imbalance, with significantly fewer cracked images, leading to biased models. To address this, we propose an integrated framework combining Generative Adversarial Networks (GANs) for minority class augmentation, a ResNet50-based classifier for crack detection, and a U-Net model for pixel-level crack segmentation. GANs generate realistic cracked surface images to mitigate data imbalance, while the ResNet50 and U-Net architectures provide robust classification and precise localization, respectively. This pipeline offers a comprehensive solution for automated structural health monitoring, particularly in data-scarce environments.

Index Terms—Concrete crack detection, Generative Adversarial Networks, ResNet50, U-Net, image segmentation, structural health monitoring, data augmentation

I. INTRODUCTION

Early detection of surface cracks is essential for ensuring the safety and longevity of civil infrastructure. Traditional manual inspections are subjective and error-prone, whereas deep learning offers a scalable and consistent alternative. However, real-world datasets often suffer from class imbalance, with far fewer cracked images, leading to biased models.

To address this, we propose an integrated framework comprising three components: (1) Generative Adversarial Networks (GANs) for minority class augmentation, (2) a ResNet50-based classifier for binary crack detection, and (3) a U-Net model for pixel-level crack segmentation. GANs generate realistic cracked surface images, mitigating data imbalance, while the ResNet50 and U-Net architectures provide robust classification and precise localization, respectively. This pipeline offers a comprehensive solution for automated structural health monitoring, particularly in data-scarce environments.

The remainder of this paper is organized as follows. Section II describes the GAN framework used for data augmentation. Section III presents the ResNet50-based classification

model. Section IV details the U-Net segmentation model. Section V concludes the paper.

II. GENERATIVE ADVERSARIAL NETWORKS FOR DATA AUGMENTATION

A. Motivation

In computer vision tasks, the performance of deep learning models is heavily reliant on the availability of large and diverse datasets. In many practical scenarios—particularly in structural health monitoring and defect detection—labeled training data is severely limited. In our case, the dataset exhibited significant data scarcity, impeding the development of a robust image classification model for crack detection.

To overcome this challenge, we employed Generative Adversarial Networks (GANs) to augment the dataset by synthesizing realistic cracked surface images, thereby enriching the training distribution.

B. GAN Framework Overview

A Generative Adversarial Network (GAN) is a class of deep generative models comprising two neural networks—the Generator (G) and the Discriminator (D)—trained in an adversarial setup:

- **Generator (G):** Attempts to produce realistic synthetic images from random noise vectors.
- **Discriminator (D):** Attempts to distinguish real images from those generated by G .

This adversarial process drives both networks to improve simultaneously: the Generator learns to produce increasingly realistic images, while the Discriminator becomes more adept at classification.

C. Hyperparameters

The GAN was trained with the configuration summarized in Table I.

TABLE I: GAN Training Hyperparameters

Parameter	Value
Number of workers	2
Batch size	64
Image spatial size	64×64
Number of channels (n_c)	3
Latent vector size (n_z)	100
Generator feature map size	64
Discriminator feature map size	64
Number of epochs	60
Learning rate	0.0002
Adam β_1	0.5
Number of GPUs	1

D. Image Preprocessing

Prior to training, all input images underwent the following preprocessing steps:

- 1) Resized to 64×64 pixels.
- 2) Center-cropped for spatial consistency.
- 3) Converted to tensor format.
- 4) Normalized with mean (0.5, 0.5, 0.5) and standard deviation (0.5, 0.5, 0.5) to standardize pixel values to the range $[-1, 1]$.

Data loading was performed using `torch.utils.data.DataLoader` with batch size 64, shuffling enabled, and 2 worker processes.

E. Weight Initialization

Weight initialization followed the standard GAN procedure:

- **Convolutional layers:** Initialized with a normal distribution ($\mu = 0$, $\sigma = 0.02$).
- **Batch normalization layers:** Weights set to 1, biases to 0.

F. Generator Architecture

The Generator transforms latent noise vectors into high-fidelity synthetic images via five transposed convolutional (`ConvTranspose2d`) layers, each followed by Batch Normalization and ReLU activation. A final Tanh activation constrains pixel values to $[-1, 1]$.

- **Input:** Noise vector of size $n_z = 100$
- **Output:** RGB image of dimensions $3 \times 64 \times 64$

G. Discriminator Architecture

The Discriminator classifies images as real or synthetic via five convolutional (`Conv2d`) layers, each followed by Batch Normalization and LeakyReLU activation. A final Sigmoid activation produces a scalar probability in $[0, 1]$.

- **Input:** RGB image of dimensions $3 \times 64 \times 64$
- **Output:** A single probability value representing image authenticity

H. Loss Functions

The standard minimax GAN loss functions were used:

$$\mathcal{L}_D = -[\log D(x) + \log(1 - D(G(z)))] \quad (1)$$

$$\mathcal{L}_G = -\log(D(G(z))) \quad (2)$$

I. Training Procedure

Training alternated between updating the Discriminator and the Generator over 60 epochs using the Adam optimizer ($\text{lr} = 0.0002$, $\beta_1 = 0.5$, $\beta_2 = 0.999$):

- 1) **Train Discriminator:** Compute real loss on genuine images, compute fake loss on generated images, and update D on total loss.
- 2) **Train Generator:** Generate images from noise, compute loss based on D 's classification, and update G to improve realism.
- 3) **Logging:** Track losses, $D(x)$, and $D(G(z))$ per epoch; store generated samples for visual inspection.

J. GAN Results

The GAN successfully generated high-quality synthetic cracked surface images (Fig. 1), effectively mitigating the effects of data scarcity. These synthesized images were combined with the original dataset to form an expanded training set used by downstream models.

III. CRACK DETECTION VIA RESNET50 CLASSIFICATION

A. Dataset

The dataset consisted of labeled images of cracked and non-cracked concrete at a resolution of 227×227 pixels in RGB JPEG format. To overcome data scarcity and enhance training diversity, GAN-generated cracked images (Section II) were combined with the original samples to form an expanded training set.

B. Preprocessing and Data Loading

The following pipeline was applied:

- 1) **Augmentation:** An `ImageDataGenerator` was configured with standard augmentation techniques (rotation, flipping, etc.) to artificially expand the dataset and mitigate overfitting.
- 2) **Preprocessing:** The `preprocess_input` function from `tensorflow.keras.applications.ResNet50` was applied, converting RGB to BGR and centering pixel values relative to the ImageNet dataset statistics.
- 3) **Validation split:** 20% of training data was held out for validation.
- 4) **Test generator:** A separate `ImageDataGenerator` without augmentation was used for the test set to maintain evaluation integrity.
- 5) **Image size:** All images were resized to 100×100 pixels in RGB mode.
- 6) **Class mode:** Set to `categorical` for binary classification (cracked vs. non-cracked).
- 7) **Batch size:** 64 for training; shuffling disabled for validation and testing.

C. Model Architecture

A transfer learning approach was adopted using ResNet50:

- 1) **Base model:** Pre-trained ResNet50 with `weights='imagenet'`, `input` shape

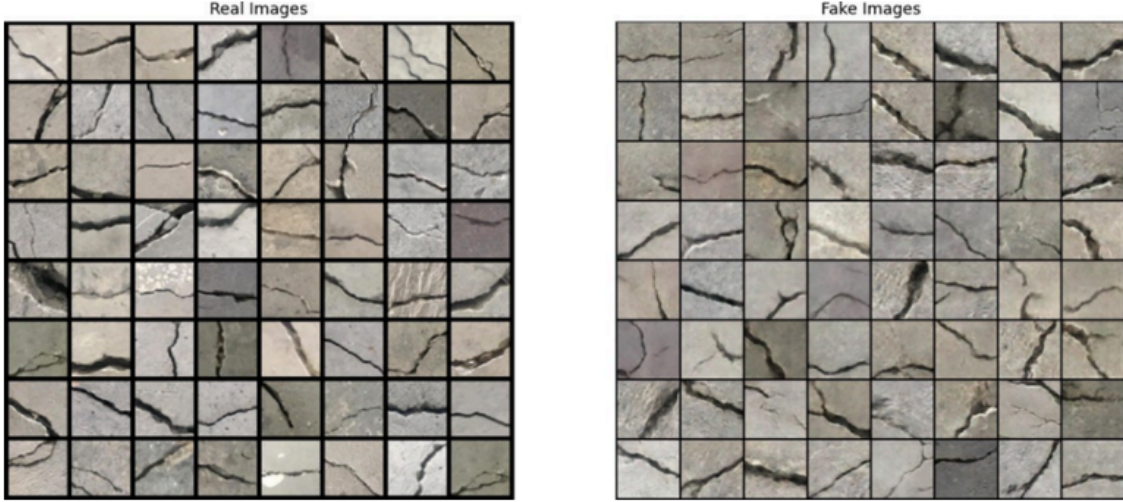


Fig. 1: Representative synthetic cracked concrete surface images generated by the GAN after training.

(100,100,3), and top classification layers excluded (include_top=False).

- 2) **Frozen backbone:** All ResNet50 layers were frozen to retain pre-trained feature representations and reduce training time.
- 3) **Classification head:** Two Dense layers with ReLU activation, followed by a Softmax output layer with 2 units.
- 4) **Compilation:** Categorical cross-entropy loss, Adam optimizer, accuracy metric.
- 5) **Early stopping:** Monitored validation loss with patience = 1.

D. Training

The model was trained for up to 100 epochs using `model.fit()`, utilizing both the original and GAN-generated images.

E. Results

1) *Training Performance:* Table II reports the per-class classification metrics on the training dataset (602 samples total).

TABLE II: ResNet50 Classification Report on Training Dataset

Class	Precision	Recall	F1-Score
0 (Non-cracked)	0.99	1.00	1.00
1 (Cracked)	1.00	0.99	1.00
Accuracy			1.00
Macro avg	1.00	1.00	1.00
Weighted avg	1.00	1.00	1.00

2) *Generalization on External Datasets:* To evaluate generalization capability, the trained model was assessed on two external datasets. Results are summarized in Table III.

TABLE III: ResNet50 Evaluation on External Datasets

Dataset	Accuracy	F1 Score	ROC
hesighsrikar/concrete-crack-images	99.33%	0.99	0.99
aniruddhsharma/structural-defects-network	82.02%	0.77	0.59

The reduced performance on the second dataset is attributed to the presence of mislabeled or ambiguous images. Nonetheless, the model retained a reasonable level of predictive power, demonstrating strong generalization from the combined training set of original and GAN-generated images.

IV. CRACK SEGMENTATION VIA U-NET

A. Dataset

Cracked concrete surface images from both the GAN-generated and original datasets were used for segmentation. A manually annotated crack image database was employed, with an 80/20 train-test split.

B. Preprocessing

- 1) **Resizing:** Images were resized to 240×160 pixels to improve computational efficiency while retaining sufficient detail.
- 2) **Normalization:** Pixel values were divided by 255 to bring them into $[0, 1]$.
- 3) **Mask processing:** The alpha channel was extracted from PNG masks, converted to grayscale, and binary-thresholded to produce binary segmentation masks.

C. U-Net Architecture

U-Net is a fully convolutional architecture originally proposed by Ronneberger et al. (2015) for biomedical image segmentation [1]. Its distinctive U-shaped structure comprises two symmetric paths:

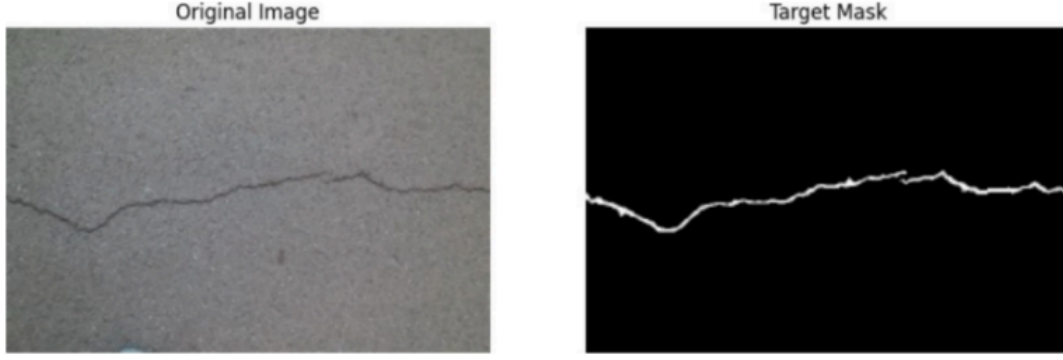


Fig. 2: Example input image and corresponding binary crack mask from the segmentation dataset.

a) *Contracting Path (Encoder)*: Successive convolutional blocks reduce spatial resolution while increasing feature depth, capturing increasingly abstract representations of the input.

b) *Expanding Path (Decoder)*: Upsampling layers progressively restore spatial resolution while reducing channel depth. *Skip connections* from the corresponding encoder levels concatenate spatial information to aid precise localization.

c) *Output*: A pixel-wise segmentation map in which each pixel is assigned a binary label (crack / non-crack).

3) *Loss and Evaluation Metrics*:

- **Loss function**: Dice coefficient loss
- **Optimizer**: Adam
- **Metrics**: Dice coefficient, Mean Intersection over Union (Mean IoU)

E. Results

1) *Training Set Performance*: Table V summarizes the U-Net metrics recorded over the final training epochs (95–100 of 100).

TABLE V: U-Net Training Metrics (Final Epochs)

Epoch	Loss	Dice Coeff.	Mean IoU
95	0.2493	0.7501	0.6190
96	0.2512	0.7503	0.5982
97	0.2500	0.7503	0.6193
98	0.2671	0.7323	0.6174
99	0.2630	0.7366	0.6048
100	0.2472	0.7526	0.6125

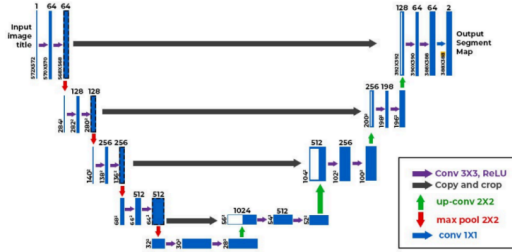


Fig. 3: U-Net architecture showing the contracting path (encoder), bottleneck, expanding path (decoder), and skip connections.

D. Model Configuration

1) *Libraries*: TensorFlow, OpenCV, NumPy, Pandas, scikit-learn.

TABLE IV: U-Net Training Hyperparameters

Parameter	Value
Batch size	8
Learning rate	10^{-2}
Training split	80%
Number of epochs	100
Early stopping patience	30
Monitored metric	Dice coefficient

2) *Hyperparameters*:

2) *Test Set Performance*: Table VI reports the U-Net metrics evaluated on the held-out test set (142 steps).

TABLE VI: U-Net Evaluation on Test Dataset

Loss	Dice Coefficient	Mean IoU
0.2856	0.6977	0.6185

3) *Qualitative Results*: The U-Net model accurately delineated crack regions in test images, as illustrated in Fig. 4.

V. CONCLUSION

This paper presented an integrated deep learning framework for automated concrete crack detection and localization. Three complementary components were developed and evaluated:

- 1) **GAN-based augmentation**: A DCGAN was trained to synthesize realistic cracked concrete images, effectively mitigating the class imbalance in the original dataset.

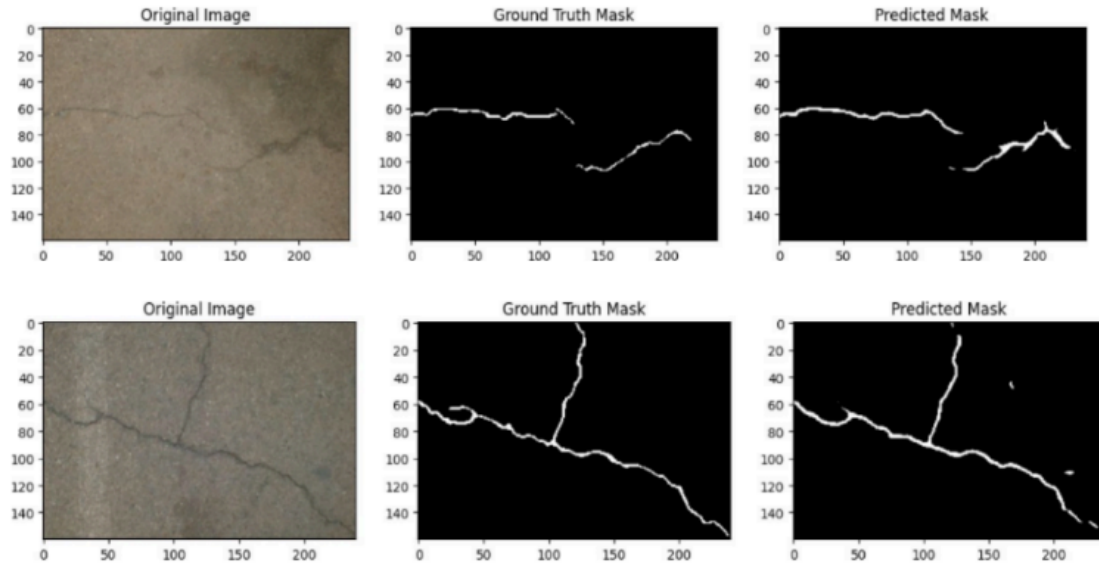


Fig. 4: Representative qualitative results of U-Net crack segmentation on test images. From left to right: input image, ground truth mask, and predicted mask.

- 2) **ResNet50 classification:** Transfer learning from ImageNet enabled robust binary classification of cracked vs. non-cracked concrete, achieving 99.33% accuracy on one external benchmark and 82.02% on a noisier dataset.
- 3) **U-Net segmentation:** Pixel-level crack delineation was achieved using a U-Net trained with Dice coefficient loss, yielding promising qualitative and quantitative results on the held-out test set.

The combined framework demonstrates that GAN-based augmentation can meaningfully improve model robustness in data-scarce structural inspection scenarios. Future work should address crack segmentation across diverse environmental and structural contexts, as well as investigate the impact of annotation quality on segmentation performance.

REFERENCES

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Proc. International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015.
- [2] I. Goodfellow *et al.*, “Generative Adversarial Nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.